

МЕТОДОЛОГИИ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ: АНАЛИЗ И КЛАССИФИКАЦИЯ

Д.А. ГУРЬЯНОВ, Г.И. АФАНАСЬЕВ, А.Г. АФАНАСЬЕВ

*ФГБОУ ВО «Московский государственный технический университет
имени Н.Э. Баумана (национальный исследовательский университет)»,
г. Москва*

Ключевые слова и фразы: жизненный цикл; методологии разработки программного обеспечения; гибкая разработка; классификация; инновационный менеджмент.

Аннотация: В статье рассмотрены основные модели жизненного цикла (ЖЦ) и методологии разработки программного обеспечения, созданные на их основе. Определено место методологий и их роль в процессе управления разработкой программного обеспечения, выявлены различия и отношения между ними, а также степень влияния на успешность реализации проекта. В работе использованы общенаучные методы исследования. Проведен обзор критериев сравнения методологий разработки ПО. Проанализированы существующие способы однокритериальной и двухкритериальной классификации методологий и выявлены их недостатки. Представлены новые способы однокритериальной классификации, устраняющие выявленные недостатки. Разработан новый вариант многокритериальной классификации на основе четырех критериев, предлагающий более полное и широкое иерархическое распределение моделей жизненного цикла и методологий разработки программного обеспечения. Рассмотрены существующие методы выбора методологии разработки программного обеспечения, а также представлен собственный метод, основанный на ретроспективном анализе и методах машинного обучения.

Введение

Процесс управления моделью жизненного цикла способствует тому, чтобы гарантировать пригодность модели, процессов и практик ЖЦ для использования организацией [1]. Этот процесс обеспечивает успешный выбор, реализацию, поддержку и адаптацию процессов ЖЦ, а также применение обоснованных методов и набора инструментов. Одним из способов решения задачи организации процессов на предприятии является выбор одной или нескольких готовых методологий разработки и развертывание процессов согласно руководствам по внедрению конкретных методологий.

Тем не менее, ввиду нематериального характера предмета разработки, процесс разработки программного обеспечения связан с высокой долей риска не завершить разработку проекта в срок. Согласно исследованиям *Standish Group* [2], по состоянию на 2020 г.,

только 31 % проектов в области разработки программного обеспечения признаны успешными. Одним из ключевых факторов [3], влияющих на высокий процент провальных проектов и завершенных с затруднениями, является неправильный выбор методологии разработки при запуске проекта.

Правильный выбор методологии на ранних этапах развития проекта является одним из ключевых мероприятий, способствующих успешности реализации проекта. На сегодняшний день у менеджеров проекта имеется более 50 возможных вариантов выбора методологии, которые различаются между собой используемой моделью жизненного цикла, ценностной ориентацией, адаптацией к размерам проекта, сроками поставки целевого продукта, составом практик разработки и пр. Такое большое количество вариантов выбора в совокупности с отсутствием достаточного количества литературных источников, слабым уровнем покрытия

(литературные источники концентрируются только на нескольких из большого числа доступных вариантов) и низким уровнем формализации процесса определения наиболее актуальной методологии, когда ее выбор осуществляется только на основании предыдущего опыта менеджера проекта и ведет к увеличению риска неправильного выбора методологии, что, в свою очередь, уменьшает вероятность успешного завершения проекта [2; 3].

Так как для решения задачи выбора методологии разработки необходимо понимать сходства и различия, в данной статье мы рассмотрим существующие методологии разработки ПО, рассмотрим их ключевые характеристики, изучим существующие методы классификации и разработаем новые способы классификации, что позволит улучшить процедуру выбора оптимальной методологии и адаптировать ее под конкретные условия, действующие на предприятии, что, в свою очередь, поможет оптимизировать весь процесс управления моделью жизненного цикла в контексте предприятия.

Литературный обзор

Вопрос выбора правильного подхода к организации жизненного цикла разработки программного обеспечения стоит перед исследователями с самых первых лет существования такой дисциплины как программная инженерия. Одна из ключевых работ, посвященных этой теме, в которой оцениваются недостатки использования каскадного подхода, применяемого некоторыми практиками, где предлагается модифицированная каскадная модель жизненного цикла, была опубликована У. Ройсом [4]. Работы Х. Мооза и К. Форсберга [5; 6], Б.В. Боема [7] содержат результаты дальнейших исследований возможности использования каскадного подхода и модифицированной каскадной модели на практике, где авторы предложили новые модели жизненного цикла разработки программного обеспечения.

Дальнейшее развитие проблема моделей ЖЦ ПО получила в работах К. Швабера [8], Дж. Стэплтона [9], К. Бека [10] и С.В. Эмбера [11], посвященным гибким методологиям разработки программного обеспечения, возникшим на рубеже тысячелетий и бросившим вызов традиционным подходам к разработке программного обеспечения. Публикация манифеста гибкой разработки привела к появлению

большого количества методологий, описанных в работе В.М. Фонта [12], и привела к проблеме выбора наиболее эффективной в зависимости от ситуации, описанной в работе А. Кокберна [13]. Процедура выбора методологии дополнительно осложняется тем, что она чаще всего не формализована в рамках конкретной организации [3]. В. Озтюрк [3] в своей статье описал решение этой проблемы с применением многокритериальных экспертных методов, таких как метод анализа иерархий [14], *MAUT* [15], *ELECTRE* [16] или вербальный анализ решений (*ВАР*) [17].

Несмотря на высокое качество результатов работы, эти методы требуют серьезных затрат времени и серьезной подготовки руководителя проекта для принятия решения. Чтобы обойти эти ограничения, были придуманы специализированные методы принятия решений выбора методологии разработки ПО, описанные в работах В. Озюрка [3] и А. Диллмана [18]. В следующих параграфах представлен краткий обзор основных идей этих методов:

А. Экспертные системы на основе правил

Данные системы представляют собой программную реализацию алгоритма, состоящего из набора вложенных условных операторов [3]. Во время выполнения процедуры выбора методологии разработки ПО пользователю необходимо ответить на последовательно задаваемые вопросы, ответы на которые влияют на выбор ветви алгоритма. В качестве недостатков данного метода можно выделить плохую приспособленность к выбору из близкородственных вариантов методологий, таких как методологии семейства *Crystal*, где основным критерием выбора из восьми вариантов является нестрогое ограничение по количеству участников в команде разработки.

В. Куб принятия решений Диллмана

В своей работе А. Dillman [18] предложил способ выбора методологии на основе графической модели расположения вариантов выбора в трехмерном пространстве. В качестве исходной идеи он использовал треугольник управления проектами *PMBoK*, который описывает идею тройственной ограниченности проекта и баланса между содержанием проекта (функциональностью), стоимостью (бюджетом) и сроками реализации (временем). Каждая из сторон треугольника управления преобразовывается в пространство трехмерной системы координат, в которой размещаются возможные варианты

выбора. Менеджеру проекта остается выбрать вариант, находящийся в интересующей его точке пространства. Данный метод позволяет принимать решения в условиях высокой неопределенности и отсутствия данных о предыдущих реализациях аналогичных проектов у команды разработки, однако рассматривает гибкие методологии в качестве единого целого. Тем не менее существенные различия в составе практик и ценностей гибких методологий могут повлиять на каждый из факторов успешности реализации проекта.

С. Выбор методологии на основе нечеткой логики

В своей работе *V. Ozturk* и коллеги [3] представили способ, который на основании таких характеристик проекта, как «Размер проекта», «Опыт команды», «Сложность», «Стабильность требований», «Риски проекта», может осуществить выбор модели жизненного цикла при помощи модели нечеткой логики.

Данная модель основывается на правилах, разработанных в результате проведения сравнительного анализа возможных вариантов выходных значений и весах правил, устанавливаемых в ходе работы менеджеров проекта индивидуально. Данная модель может эффективно предсказывать удачную методологию на самых ранних этапах проекта (даже до его начала), так как в качестве входных параметров используются лингвистические переменные, представленные в виде отдельных слов или фраз, описывающих различные характеристики качества проекта на понятном человеку языке в виде степеней уверенности и относительных величин. В таком виде данные о проекте понятны менеджеру еще до запуска проекта. Данный способ может быть масштабирован для принятия решения о выборе конкретной методологии, однако их число будет ограничено границами применимости модели нечеткой логики. Согласно рекомендации автора метода, допустимое число правил, при которых модель может дать адекватный результат, составляет не более 15, что не позволит описать все необходимые отличия близкородственных гибких методологий.

Большое количество вариантов побудило исследователей изучить детали и выявить общие черты сравнения, которые могли бы лечь в основу классификации методологий. В своих статьях *D. Ciric et al.* [19] предложили варианты классификации на основе одного критерия – гибкость процесса, а *Z. Ahmed et al.* [20]

предоставили свои варианты в зависимости от размера команды. *М. Фаулер* [21] предложил вариант двухкритериальной классификации, основанный на наложении двух проекций одно-критериальных классификаций в виде двумерной системы координат.

Аналогичное исследование было опубликовано *А. Кокберном* [13], где в качестве критериев были выбраны размер команды и риски проекта. Такие классификации обычно используются как самостоятельные инструменты для выбора методологии разработки программного обеспечения или для уменьшения количества нерелевантных вариантов в комплексном подходе.

Эти идеи более подробно обсуждаются далее в этой статье.

Жизненный цикл разработки программного обеспечения

Период существования программной системы, который начинается с момента замысла, продолжается в течение эволюции и заканчивается утилизацией программной системы называется жизненным циклом (**ЖЦ**) программной системы [22]. В качестве концептуальной основы, используемой для организации и управления разработкой, функционированием, сопровождением и утилизацией программных систем, были разработаны модели жизненного цикла программных систем. Большинство из них многократно детально изучались, в данной статье основное внимание будет уделено следующим моделям:

- неструктурированная разработка («взрывное проектирование») [4];
- анализ-кодирование [4];
- каскадная (*Waterfall*) [4; 12];
- модифицированная каскадная [4; 12];
- *V*-образная [5];
- *W*-образная [6];
- инкрементальная [23];
- итерационная [23];
- спиральная [7].

На протяжении реализации проекта модели жизненного цикла уточняются конкретными методологиями разработки, представляющими собой свод правил, практик, методов и инструкций, способствующих успешной реализации проекта.

Как было сказано выше, у руководителей проектов есть более 50 вариантов выбора ме-

тодологии разработки программного обеспечения, большинство из которых неоднократно подробно изучались [24; 25].

В дальнейшем для поддержания контекста обсуждения стоит выделить некоторые из наиболее популярных, по мнению авторов, методологий, которые будут упомянуты далее в этой статье. Эти методологии включают:

- *Rapid Application Development (RAD)* [12];
- *Rational Unified Process (RUP)* [26];
- *OpenUP* [24; 25];
- *Agile Unified Process (AUP)* [11];
- *Scrum* [8];
- *Large Scale Scrum (LeSS)* [20];
- *Dynamic System Development Method* [9];
- *Agile Kanban Methodology* [12];
- *Extreme Programming (XP)* [10];
- *DevOps* [12];
- *Microsoft Solutions Framework* [27].

Обзор критериев сравнения методологий разработки ПО

В качестве предварительного шага перед проведением анализа способов классификации методологий разработки ПО проведем поиск критериев, которые могут лечь в основу их классификации.

Способ декомпозиции проекта

В своей работе Мартин Фаулер [21] выявил наличие двух концептуальных подходов к построению жизненного цикла программной системы – каскадного и итерационного подходов. Различие между данными подходами заключается в декомпозиции проекта на более мелкие части. Так, в каскадном подходе применяется линейно-структурная декомпозиция согласно правилам *PMBoK*, проект проецируется на план-график, и прогресс выполнения оценивается в процентах выполнения графика работ. В итеративном подходе проект разбивается на функциональные части, каждая из которых представляет собой подпроект и имеет промежуточный результат в виде версии продукта.

Модель жизненного цикла

Несмотря на то, что стандарт *ISO 12207* явно не регулирует способы организации последовательности этапов разработки, в исследованиях Ройса [4], Форсберга и Мооза [5; 6] и других были представлены различные абстрактные модели организации последовательности этапов жизненного цикла, имену-

емые моделями жизненного цикла, рассмотренные ранее в этой статье. Существующие методологии разработки ПО реализуют одну из существующих моделей и добавляют к ней свои особенности в виде практик, артефактов, программного обеспечения и прочих инструментов, которые поддерживают процесс разработки.

Гибкость процесса

Данный показатель показывает степень формализации процесса разработки, выраженную в виде возможности возврата на предыдущий этап ЖЦ, объеме необходимых артефактов, требуемых для перехода между этапами, возможности корректировки плана разработки и участия заказчика в процессах разработки.

В работе Кокберна [13] данный показатель именуется как «методологический вес» и определяет то, какие части процесса разработки (роли участников, технологии, инструменты и пр.) регулируются методологией (и степень формализации – от устного общения до обмена информацией на бумажном носителе), а какие остаются на саморегулировании участниками команды.

Используемые практики

В современном процессе разработки ПО наблюдается тенденция на разработку новых гибридных методологий, сочетающих в себе преимущества нескольких изобретенных ранее. К таким методологиям можно отнести *eXRUP* [11], сочетающую в себе парное программирование, являющееся отличительной чертой *XP* с процессом и *CASE*-средствами из *RUP* [11].

Архитектурная сложность проекта

Данный критерий описывает наличие этапа проектирования в процессе ЖЦ и количество уровней этого проектирования. Например, может использоваться принцип «простого дизайна» [10], применяемый в *XP*, требующий дополнительного «непрерывного рефакторинга» или противоположное ему решение в виде двух этапов проектирования, применяемого в *RUP* [11].

Размер команды

Данный критерий представлен в работе Кокберна [13] и описывает проблемы взаимодействия внутри команды проекта, появляющиеся при одновременном росте числа сотрудников без дополнительной формализации процесса общения. Критерий размера команды является одним из факторов выбора конкретной методологии в семействе методологий *Crystal*. Этим вопросом занимаются и другие

Таблица 1. Фрагмент финальной таблицы сравнения методологий разработки ПО

	Частичная поставка	Управление рисками	Управление требованиями	Прото-типирование	Применение CASE-средств	Повторное использование	Ежедневные стенд-апы
Взрывное проектирование							
Анализ-кодирование							
Кодирование-отладка							
Каскадная							
Модифицированная каскадная		✓	✓				
V-образная		✓	✓	✓			
W-образная		✓	✓	✓			
Спиральная	✓	✓	✓	✓			
Эволюционное прототипирование	✓	✓	✓	✓	✓		
RAD	✓	✓	✓	✓	✓	✓	
XP	✓	✓	✓	✓			✓
DSDM	✓	✓	✓	✓			
Scrum	✓	✓	✓	✓			✓

авторы гибких методологий путем введения ограничения на размер команды, как в *Scrum*, и разработки специальных фреймворков для управления большими проектными командами, например, *LeSS*.

Приоритеты и принципы

Совпадение приоритетов проекта и приоритетов методологии является одним из факторов, используемых при выборе конкретной методологии разработки.

Так, например, выбор методологии, ориентированной на разработку инструментов поддержки и автоматизации процесса, может сократить сроки выпуска новых версий продукта, а использование командно-ориентированных методологий повышает скорость реакции на меняющиеся требования и повышает удовлетворенность заказчика [28].

К сожалению, полные результаты сравнения методологий не могут быть размещены в данной статье из-за значительного размера результирующей таблицы.

Часть этой таблицы представлена в табл. 1. Окончательный вариант таблицы, включающий 48 методологий и 15 критериев, можно обнаружить в [47].

Обзор существующих способов классификации

Большое число вышеописанных критериев оценки, влияющих на характеристики методологий, затрудняет возможность построения единой строгой классификации, так как методологии, различные по нескольким важным признакам, таким как гибкость процесса или модель жизненного цикла, могут оказаться схожими по другим не менее важным признакам, таким как адаптация к размеру проекта и приоритеты продукта. Тем не менее были разработаны и успешно применяются классификации, учитывающие один или два из критериев сравнения.

Далее мы рассмотрим существующие классификации методологий разработки программного обеспечения.

Особое внимание обратим на тот факт, что классификации, представленные далее, как и большинство литературных источников, не делают различие между понятиями модели жизненного цикла и методологии разработки. В этом разделе они объединяются под одним термином – методология.

Таблица 2. Традиционные/гибкие методологии

Традиционные	Гибридные	Гибкие
<i>Waterfall, Vee Model, Double Vee</i>	<i>RUP, RAD, MSF, Спиральная</i>	<i>SCRUM, Kanban, DSDM, XP, Анализ-кодирование, DevOps, AUP, Взрывное проектирование</i>

Таблица 3. Классификация по размеру команды

Для малых команд	Для больших команд
Анализ-кодирование, <i>Scrum, RAD, DSDM, XP, Kanban, AUP, DevOps</i> , Взрывное проектирование	<i>RUP, Waterfall, MSF, Vee Model, Double Vee</i> , Спиральная

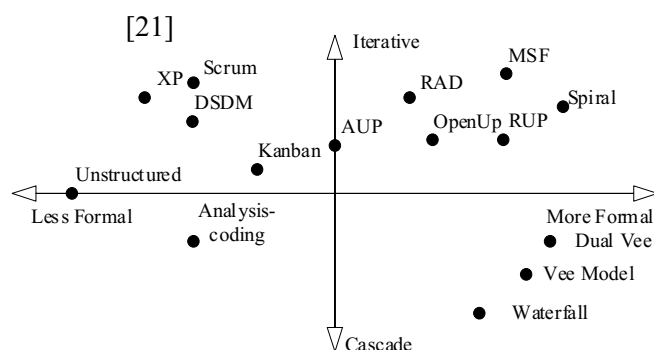


Рис. 1. Схема двухкритериальной классификации методологий разработки ПО

Классификация гибкости процесса

Традиционная классификация, рассматривающая методологии с точки зрения необходимости строго следовать порядку выполнения этапов жизненного цикла и приоритетам проекта. В случае *process-first/documentation-first* подхода (Приоритет документации и процесса) – методология относится к классу традиционных, в случае подхода *people-first/product-first* (Приоритет команды и продукта, соответствие принципам манифеста гибкой разработки) – к классу гибких. В случаях, когда методология разработана с целью учесть оба возможных подхода к разработке ПО, – к классу гибридных [19]. В табл. 2 представлены примеры методологий, относящихся к каждому классу.

Классификация по размеру команды

Данная классификация предполагает разделение методологий по допустимости их применения в больших и малых командах. Например, методология *Scrum* спроектирована для работы кросс-функциональной команды, состоящей максимум из 12 человек. В случае необходи-

мости применения принципов *Scrum* для более больших продуктовых команд, существуют специальные методологии, такие как *Large Scale Scrum (LeSS)*, учитывающие особенности работы в больших командах [20]. В табл. 3 представлены примеры методологий, относящихся к каждому классу.

Двухкритериальная классификация

Данный способ классификации основан на наложении двух проекций однокритериальных классификаций на числовые линии в виде двумерной системы координат. Таким образом, оценка каждой из методологий получает координатную оценку и размещается на координатной плоскости в виде точки. Такой способ классификации позволяет визуально оценить методологии по двум критериям одновременно, и при необходимости использовать вычисление длины вектора как способ выбора предпочтительной методологии разработки [23]. На рис. 1 представлен пример подобной классификации, основанный на критериях способа декомпозиции проекта и гибкости процесса.

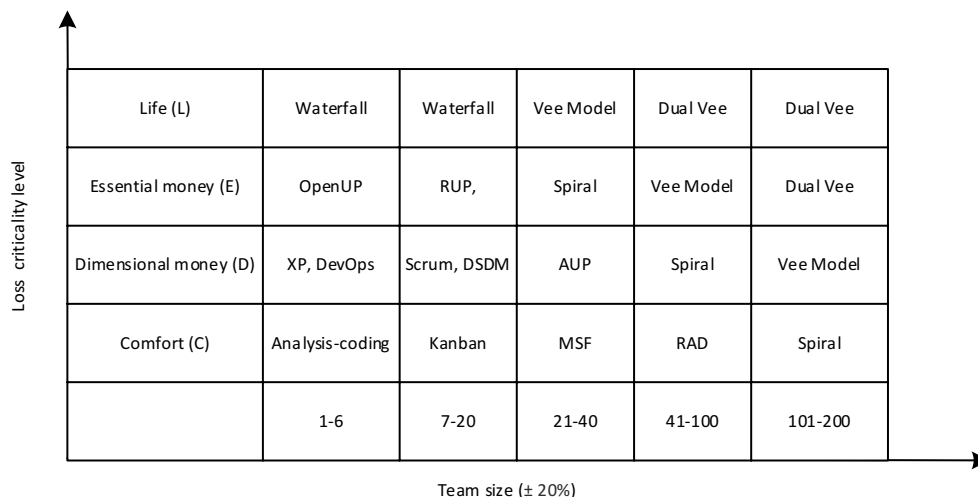


Рис. 2. Классификация по шкале Кокберна

Таблица 4. Классификация по моделям жизненного цикла

Метод декомпозиции	Модель ЖЦ	Семейство Методологий
Каскадный	Каскадная «Взрывное проектирование», Анализ-кодирование, Водопадная	—
	V-образная <i>Vee Model, Dual Vee</i>	—
Итеративный	Инкрементная	—
	Итеративная	Унифицированный процесс <i>RUP, OpenUP</i>
		Гибридные <i>RAD, AUP, MSF</i>
		Гибкие <i>XP, Scrum, DSDM, DevOps</i>

Шкала Кокберна

Данная классификация предложена Алистером Кокберном [13] и основана на матричном представлении классификации по размеру команды и шкалы критичности, описывающей риски проекта: риск потери комфорта (*C*), риск потери дополнительного дохода (*D*), риск банкротства (*E*) и риск потери жизни (*L*). Чем выше уровень критичности, тем более формализованная требуется методология разработки. Методологии размещаются в ячейках таблицы в соответствии со своей применимостью к конкретным размерам команды и критичностью проекта, на рис. 2 представлено распределение рассмотренных в данной статье методологий по классам шкалы Кокберна.

Предлагаемые способы классификации

Ввиду того что вышеописанные способы классификации обладают такими недостатками, как малое количество анализируемых критериев и избыточно большие размеры кластеров, из-за чего затруднен выбор наиболее подходящей методологии [24; 25], нами были разработаны новые способы классификации методологий разработки ПО, которые представлены далее.

Классификация по моделям жизненного цикла

Эта классификация основана на критериях способа декомпозиции проекта и выборе модели жизненного цикла и представляет собой дерево классов, каждый уровень которого пред-

Таблица 5. Иерархическая классификация моделей ЖЦ

СОПРОВОЖДАЕМОСТЬ (стабильность требований)	НАДЕЖНОСТЬ	ФУНКЦИОНАЛЬНОСТЬ (сложность) <i>Модель ЖЦ</i>	ОРИЕНТАЦИЯ (приоритет методологии) <i>Методология</i>
ЖЕСТКИЕ (строгие требования)	ВЫСОКОЙ НАДЕЖНОСТИ	ВЫСОКОЙ СЛОЖНОСТИ <i>W-образная</i>	
		СЛОЖНЫЕ <i>V-образная</i>	
	НАДЕЖНЫЕ	ВЫСОКОЙ СЛОЖНОСТИ Инкрементная	
		СЛОЖНЫЕ <i>Каскадная</i>	
ГИБРИДНЫЕ (требования изменяются)	НАДЕЖНЫЕ	СЛОЖНЫЕ <i>Итеративная, Спиральная</i>	Процессно-ориентированные <i>MSF</i>
			Объектно-ориентированные <i>RUP</i>
ГИБКИЕ (требования постоянно уточняются)	СРЕДНЕЙ НАДЕЖНОСТИ	СЛОЖНЫЕ <i>Итеративная</i>	Процессно-ориентированные <i>AUP, eXRUP</i>
		ПРОСТЫЕ <i>Итеративная</i>	Инструментальные <i>RAD, Kanban, DevOps, LeSS, OpenUP</i>
	НИЗКОЙ НАДЕЖНОСТИ	ПРОСТЫЕ <i>Взрывное проектирование, Анализ-кодирование, Кодирование-отладка</i>	Командные <i>XP, Scrum, DSDM</i>

ставляет собой декомпозицию предыдущего уровня по определенному критерию:

а) уровень способа декомпозиции проекта, на котором происходит деление на линейные и циклические согласно идеям Фаулера [21], описанным выше;

б) уровень модели жизненного цикла;

с) уровень семейства методологий, который существует только для итерационной модели ЖЦ, и на нем производится группировка методологий в семейства на основании использования в качестве основы *Agile* или унифицированного процесса *UP*;

д) уровень методологий.

Распределение основных известных методологий разработки программного обеспечения и схема классификации по моделям ЖЦ представлены в табл. 4.

Иерархическая классификация

Предлагаемая классификация представляет собой дерево классов, каждый уровень которого представляет собой одну из важных характеристик качества проекта и направленности методологии.

Распределение основных известных методологий разработки и схема их классификации представлены в табл. 5.

По характеристике проекта СОПРОВОЖДАЕМОСТЬ (уровень стабильности требований) методологии разделены на три класса: ЖЕСТКИЕ – требования к проекту полностью определены и в процессе реализации могут лишь уточняться; ГИБРИДНЫЕ – требования к проекту частично определены, а в процессе реализации изменяются; ГИБКИЕ – требования к проекту уточняются в процессе его реализации.

По характеристике проекта НАДЕЖНОСТЬ методологии разделены на следующие классы: ВЫСОКОНАДЕЖНЫЕ; НАДЕЖНЫЕ; СРЕДНЕЙ НАДЕЖНОСТИ; НИЗКОЙ НАДЕЖНОСТИ.

По характеристике проекта ФУНКЦИОНАЛЬНОСТЬ (уровень сложности ПО, на этом же уровне определены модели ЖЦ, на которых базируются методологии) методологии разделены на три класса: ВЫСОКОЙ СЛОЖНОСТИ; СЛОЖНЫЕ; ПРОСТЫЕ.

По ОРИЕНТАЦИИ проекта (приоритетам и принципам) методологии разделены на следующие группы: ПРОЦЕССНО-ОРИЕНТИРОВАННЫЕ – ориентированные на структуру процессов (фазы) ЖЦ ПО; ОБЪЕКТНО-ОРИЕНТИРОВАННЫЕ – ориентированные на сценарии процессов ЖЦ ПО и объектно-ориентированные методы разработки; ИНСТРУМЕНТАЛЬНЫЕ – ориентированные на команды разработчиков, активно использующие инструменты автоматизации разработки, тестирования и развертывания ПО; КОМАНДНЫЕ – ориентированные на небольшие команды разработчиков, для которых программный продукт важнее процессов и документации проекта.

Выбор методологии разработки программного обеспечения

Для принятия решения о выборе конкретной методологии разработки ПО можно использовать многокритериальные экспертные методы, которые можно разделить на следующие группы с точки зрения измерений:

- основанные на количественных измерениях (многокритериальная теория полезности – *MAUT* [15]);
- основанные на качественных результатах, переведенные в количественный вид (метод анализа иерархий – *AHP* [14], экспертные системы на основе правил [3], методы на основе нечеткой логики [3, с. 29–32]);
- основанные на количественных измерениях с использованием нескольких индикаторов при сравнении альтернатив (семейство методов *Electre* [16]);
- основанные на качественных характеристиках без перевода в количественный вид (куб принятия решений Диллмана [18], вербальный анализ решений – *ВАР* [17]).

В качестве альтернативы рассмотренным

выше способам выбора методологии был разработан метод, основанный на ретроспективном анализе исторических данных о ранее завершенных проектах. Основная идея этого метода заключается в анализе исходных данных о проекте при помощи методов машинного обучения, прогнозировании выходных метрик планируемого проекта и вычислении подходящей методологии на основе интегральной оценки прогнозируемых результатов для каждого доступного варианта выбора методологии. На рис. 3 представлена схема предлагаемого алгоритма выбора методологии. В качестве исходной информации используются данные о ранее реализованных командой (группой, отделом, компанией) проектах: метрики предварительной оценки проекта, метрики итоговой (фактической) оценки и использованной методологии.

В качестве метода предварительной оценки при планировании разработки ПО используются модели оценки стоимости разработки программного обеспечения, такие как *ESTIMACS* или *COCOMO-81*. Последняя, одна из самых популярных моделей, была предложена в 1981 г. *B. Boehm* [33] и основывается на анализе 63 проектов, разработанных компанией *TRW* в период с 1964 по 1979 гг. [33–34]. Несмотря на то, что вышеописанные исследования не учитывают более современные модели оценки стоимости, такие как уточненная *COCOMO II* этапа постархитектуры, выбор в пользу среднего уровня *COCOMO-81* был осуществлен из-за открытости подхода (такие модели как *ESTIMACS* являются проприетарными), а также благодаря наличию готовых наборов для обучения нейронной сети, находящихся в свободном доступе. Для оценки стоимости разработки программного обеспечения в *COCOMO-81* используются 15 метрик оценки проекта, учитывающие большое количество различных факторов, влияющих на разработку проекта (от ожидаемого размера базы данных до опыта разработки и уровня знания языка программирования у членов команды разработки). Значения данных метрик в нашем методе используются в качестве входов для нейросети, предсказывающей итоговую оценку проекта. В качестве дополнительного параметра мы будем использовать наименование методологии разработки, использованной при разработке программного проекта.

Оценка результатов разработки проводится менеджером проекта по результатам завер-



Рис. 3. Схема предлагаемого алгоритма выбора методологии

Таблица 6. Метрики итоговой оценки проекта

Имя	Наименование	Описание
<i>RTME</i>	Срок реализации	Отношение между плановым и итоговым сроком реализации. Допустимые значения находятся в диапазоне от 0 до 10 и определяют во сколько раз быстрее (или медленнее) был завершен проект
<i>RCST</i>	Стоимость разработки	Отношение между плановыми и итоговыми финансовыми затратами. Допустимые значения находятся в диапазоне от 0 до 10 и определяют во сколько раз дороже (или дешевле) обошлась разработка проекта
<i>RFNL</i>	Функциональность	Комплексная оценка на основе критерия «Функциональность», вычисленная согласно методике оценки качества программных средств согласно ГОСТ 28195-89 [43]. Допустимые значения находятся в диапазоне от 0 до 1, где 1 означает полное соответствие реализации функциональным требованиям
<i>RQLT</i>	Надежность	Комплексная оценка на основе критерия «Надежность», вычисленная согласно методике оценки качества программных средств согласно ГОСТ 28195-89 [43]. Допустимые значения находятся в диапазоне от 0 до 1, где 1 означает полное соответствие требованиям надежности

шения разработки. С точки зрения управления проектом для успешной реализации проекта необходимо сбалансировать противоречивые требования и четыре наиболее важных параметра проекта: срок реализации, стоимость разработки, функциональность и надежность [37]. Для оценки фактора успешности представим перечисленные параметры в качестве метрик оценки проекта. Описание ограничений для вводных

параметров описаны в табл. 6.

Возможность расчета перечисленных метрик менеджером на ранних этапах планирования разработки достаточно ограничена ввиду неопределенности будущего [38–42], тем не менее их предварительная оценка возможна при помощи средств машинного обучения. Таким образом, метрики итоговой оценки проекта являются выходами нейронной сети.

Таблица 7. Фрагмент итоговой таблицы результатов эксперимента

МЕТРИКА	Проект 1	Проект 2	Проект 3	Проект 4	ПРОЕКТ 5
Входные аргументы модели					
<i>RELY</i>	0,88	0,88	1	0,75	0,88
<i>DATA</i>	1,16	1,16	1,16	1,16	0,94
<i>CPLX</i>	0,7	0,85	0,85	0,7	1
<i>TIME</i>	1	1	1	1	1
<i>STOR</i>	1,06	1,06	1	1	1
<i>VIRT</i>	1,15	1	0,87	0,87	0,87
<i>TURN</i>	1,07	1,07	0,94	1	1
<i>ACAP</i>	1,19	1	0,86	1,19	1
<i>AEXP</i>	1,13	0,91	0,82	0,91	1
<i>PCAP</i>	1,17	1	0,86	1,42	0,86
<i>VEXP</i>	1,1	0,9	0,9	1	0,9
<i>LEXP</i>	1	0,95	0,95	0,95	0,95
<i>MODP</i>	1,24	1,1	0,91	1,24	1,24
<i>TOOL</i>	1,1	1	0,91	1	1
<i>SCED</i>	1,04	1	1	1,04	1
<i>MTOD*</i>	1	0,5	0	0,5	0,5
Выходные значения модели					
<i>Implementation time*</i>	0,7	1	1	0,8	0,7
<i>Development cost*</i>	0,7	1	1,1	1,2	1,2
<i>Functionality*</i>	0,85	1	0,9	1	0,95
<i>Quality*</i>	0,93	0,9	1	1	0,98
Критерий					
$\max(O_{methodology})$	1,15929	0,975	0,95225	1,02	0,9196
Результаты					
Методология предсказанная (методом)	Спиральная	Спиральная	Анализ-кодирование	Водопадная	Спиральная
Методология фактическая (ожидаемое значение)	Спиральная	Водопадная	Анализ-кодирование	Водопадная	Водопадная

* – значение параметра указано для соответствующего значение критерия $\max(O_{methodology})$

В качестве архитектуры нейронной сети была выбрана классическая четырехслойная сетевая схема многослойного перцептрона с 16 входами на первом слое для каждого из параметров проекта, двумя промежуточными слоями

и 4 выходами. В качестве функции активации промежуточных слоев используется сигмоидальная функция, функция активации выходного слоя – линейная. В исходной выборке используются 15 параметров из модели оценки

проекта *COCOMO-81*, дополнительный входной параметр *MTOD* (используемая методология) и ожидаемые значения выходных характеристик проекта *RTME*, *RCST*, *RFNL* и *RQLT*. Выбор таких выходных значений определяется основными метриками проекта, описанными в *PMBoK* и *DSDM* [37].

Для завершения процесса выбора методологии необходимо осуществить вычисление прогнозов метрик итоговой оценки для каждого из рассматриваемых вариантов выбора методологии [44–46]. После завершения этих вычислений методологии оцениваются по следующей формуле:

$$O_{\text{метод}} = (0,1 \times RTME + 0,1 \times RCST + RFNL + RQLT) / 4.$$

Выходные параметры *RTME* и *RCST* в формуле были умножены на 0,1, поскольку, как указано в таблице 6, они представляют собой соотношение между фактическим и затраченным временем и бюджетом. Использование коэффициента 0,1 помогает в этой ситуации привести значения всех параметров к одной экспоненциальной части.

Чем больше итоговый балл, тем более эффективна данная методология для данной проектной команды. В качестве рекомендуемой методологии выбирается методология с наибольшим итоговым баллом. К достоинствам данного способа выбора стоит отнести адаптируемость к конкретным ограничениям организации и предыдущему опыту проектной команды, однако в условиях отсутствия данных о предыдущем опыте работы команды, результаты предсказания на основе методов машинного обучения будут некорректными, и в таком случае до накопления данных об опыте рекомендуется использовать методы, подходящие для работы в условиях неопределенности, которые были представлены в данной статье.

Для оценки эффективности прогнозирования методологии был проведен эксперимент, где в качестве исходной выборки был выбран набор данных, предложенный *B. Boehm* [34] в 1981 г. при разработке модели *COCOMO-81*. В качестве допустимых вариантов значения параметра *MTOD* использовались: Анализ-кодирование, *Waterfall*, *Spiral*. Значения выходных метрик были вычислены вручную, после чего программой, реализующей поведение нейронной сети и проводящей вычисление значения

итоговой методологии, выполнялось прогнозирование рекомендуемой методологии для тестовой части выборки. Программная реализация нейронной сети была разработана авторами с использованием языка программирования *C#* и *.NET 5*. По итогам 10000 циклов обучения сети среднее значение ошибки составило 0,0059, а ожидаемая методология была предсказана корректно в 13 тестовых случаях из 15.

Таким образом, точность прогнозирования составляет 87 %, что может быть связано прежде всего с малыми объемами обучающей и тестовой выборки и малым числом вариантов выбора методологии. Фрагмент итоговой таблицы с результатами эксперимента представлен в табл. 7.

Заключение

Согласно исследованиям *Standish*, корректный выбор методологии разработки повышает вероятность успешного завершения проекта в области разработки ПО на 16 %.

Таким образом, умение выбирать методологию критически важно для менеджеров проекта. Были рассмотрены существующие подходы к классификации методологий, выявлены их недостатки и предложены собственные варианты, предназначенные для расширения числа рассматриваемых критериев и более точного распределения методологий по классам.

Использование новых классификаций поможет менеджерам проекта более точно выбрать методологию разработки ПО и сократить риски в процессе управления проектом.

В статье также рассмотрены существующие способы выбора методологии разработки ПО, кратко проанализированы их достоинства и недостатки, а также предложен новый способ выбора методологии на основе ретроспективного анализа и методов машинного обучения, что позволит адаптировать процедуру выбора под конкретные условия, действующие на предприятии.

К планам дальнейших работ относится исследование поведения метода на большем объеме обучающей выборки (в качестве набора данных предлагается использование набора данных разработки ИТ-проектов *NASA*), а также исследование применимости и точности предсказаний в случае замены модели с *COCOMO-81* на более современные подходы оценки стоимости разработки.

Литература/References

1. ISO/IEC/IEEE 12207:2017. Systems and Software Engineering – Software Life Cycle Processes, [Electronic resource]. – Access mode : <https://www.iso.org/ru/standard/63712.html>.
2. Review Standish Group – CHAOS 2020: Beyond Infinity [Electronic resource]. – Access mode : <https://hennyportman.wordpress.com/2021/01/06/review-standish-group-chaos-2020-beyond-infinity>.
3. Ozturk, V. Selection of Appropriate Software Development Life Cycle Using Fuzzy Logic / V. Ozturk et al. // Journal of Intelligent and Fuzzy Systems. – 2013. – Vol. 25. – P. 797–810.
4. Royce, W. Managing the development of large software systems: Concepts and techniques / W. Royce // IEEE WESCOM. IEEE Computer Society Press, Los Alamitos, 1970.
5. Mooz, H. The Relationship of System Engineering to the Project Cycle / H. Mooz, K. Forsberg // International Symposium of the International Council On Systems Engineering (INCOSE), 1991. – P. 57–65.
6. Mooz, H. The Dual Vee – Illuminating the Management of Complexity / H. Mooz, K. Forsberg // Sixteenth Annual International Symposium of the International Council On Systems Engineering (INCOSE), 2006. – P. 1368–1381.
7. Boehm, B.W. A Spiral Model of Software Development and Enhancement / B.W. Boehm // Proceedings of an International Workshop on Software Process and Software Environments. – Coto de Caza, Trabuco Canyon, California, 1985.
8. Schwaber, K. The scrum guide / K. Schwaber. – Scrum Alliance, 2011.
9. Stapleton, J. DSDM: Dynamic Systems Development Method / J. Stapleton // Proceedings Technology of Object-Oriented Languages and Systems, TOOLS 29 (Cat. no. PR00275), 1999.
10. Beck, K. Extreme Programming Explained: Embrace Change / K. Beck. – Addison-Wesley, 1999.
11. Amber, S.W. The Agile Unified Process (AUP) / S.W. Amber. – Ambysoft, 2005.
12. Font Jr., V.M. The Ultimate Guide to SDLC / V.M. Font Jr. – FontLife, 2012.
13. Cockburn, A. Selecting a project's methodology / A. Cockburn // IEEE Software. – 2000. – Vol. 17. – No. 4. – P. 64–71.
14. Saati, T.L. Decision Making under Dependencies and Feedbacks: Analytical Network / T.L. Saati. – Pittsburgh, PA : Rws Pubns, 2001.
15. Keeney, R.L. Decisions with Multiple Objectives: Preferences and Value Trade-Offs / R.L. Keeney, H. Raiffa // IEEE Transactions on Systems, Man and Cybernetics. – 1977. – Vol. 9. – No. 7. – P. 403.
16. Figueira, J. ELECTRE Methods / J. Figueira, V. Mousseau, B. Roy // International Series in Operations Research & Management Science, 2005. – P. 133–162.
17. Larichev, O.I. Method CICL for ordinal classification of multicriteria alternatives / O.I. Larichev, A.A. Asanov // Doklady Mathematics. – 2000. – Vol. 62. – No. 3. – P. 444–448.
18. Dillman, A. Selecting a Software Development Life Cycle (SDLC) Methodology / A. Dillman // PM Solutions White Paper, 2003. – P. 1–12.
19. Ciric, D. Agile vs. Traditional Approach in Project Management: Strategies, Challenges, and Reasons to Introduce Agile / D. Ciric et. al. // Proc. 25th International Conference on Production Research Manufacturing Innovation: Cyber Physical Manufacturing, 2019. – P. 1408–1414.
20. Ahmed, Z. Preference in using Agile Development with Larger Team Size / Z. Ahmed, A. Waleed, M. Waqas // International Journal of Advanced Computer Science and Applications. – 2018. – Vol. 9. – P. 116–123.
21. Fowler, M. The new methodology / M. Fowler // Wuhan Univ. J. of Nat. Sci. – 2001. – Vol. 6. – P. 12–24.
22. ISO/IEC 15704:2019. Enterprise modeling and architecture – Requirements for enterprise-referencing architectures and methodologies [Electronic resource]. – Access mode : <https://www.iso.org/obp/ui/#iso:std:iso:15704:ed-2:v1:en>.
23. Futrell, R.T. Quality Software Project Management / R.T. Futrell, D.F. Shafer, L.I. Shafer. – M. : Williams, 2003.
24. Gurianov, D.A. Classification of software development methodologies / D.A. Gurianov,

K.S. Myshenkov // Integrated Models and Soft Computing in Artificial Intelligence IMSC-2022 : Proc. of the XI International Scientific and Practical Conference. – 2022. – Vol. 1. – P. 279–292.

25. Gurianov, D.A. Analysis of software development methodologies / D.A. Gurianov, K.S. Myshenkov // International Scientific and Technical Congress “Intelligent Systems and Information Technologies-2022” : Proc. of the International Scientific and Technical Congress (IS&IT-2022). – 2022. – Vol. 1. – P. 304–310.

26. Sharma, N. eXSRUP: Hybrid Software Development Model Integrating Extreme Programming, Scrum & Rational Unified Process / N. Sharma, M. Wadhwa // TELKOMNIKA Indonesian Journal of Electrical Engineering. – 2015. – Vol. 16. – No. 2.

27. Essama Ze, S. Microsoft Solutions Framework (MSF) for Agile Software Development / S. Essama Ze, 2014.

28. Dingsoy, T. A decade of agile methodologies: Towards explaining agile software development / T. Dingsoy et al. // The Journal of Systems and Software. – 2012. – Vol. 85. – P. 1213–1221.

29. Averkin, A.N. The fuzzy modelling relation and its application in psychology and artificial intelligence / A.N. Averkin, V.B. Tarasov // Fuzzy Sets and Systems. – 1987. – Vol. 22. – P. 3–24.

30. Zadeh, L.A. Fuzzy sets / L.A. Zadeh // Information and Control. – 1965. – Vol. 8. – P. 338–353.

31. Aliev, R.A. Fundamentals of the Fuzzy Logic-Based Generalized Theory of Decisions / R.A. Aliev. – Berlin : Heidelberg : Springer-Verlag, 2013.

32. Zadeh, L.A. Fuzzy Logic for the Management of Uncertainty / L.A. Zadeh, J. Kacprzyk. – New York : John Wiley & Sons, 1992.

33. Boehm, B.W. Cost Models for Future Software Life Cycle Processes: COCOMO 2.0 / B.W. Boehm // Annals of Software Engineering. – 1995. – Vol. 12. – P. 1–31.

34. B.W. Boehm, “Software Engineering Economics”, Englewood Cliffs, N.J.: Prentice-Hall, 1981.

35. Sakulin, S. Network Anomalies Detection Approach Based on Weighted Voting / S. Sakulin et. al. // International Journal of Information Security And Privacy. – 2022. – Vol. 16. – No. 1.

36. Osipov, A. Identification and Classification of Mechanical Damage During Continuous Harvesting of Root Crops Using Computer Vision Methods / A. Osipov et. al. // IEEE ACCESS. – 2022. – Vol. 10. – P. 28885–28894.

37. The DSDM Agile Project Framework [Electronic resource]. – Access mode : <https://www.agilebusiness.org/page/TheDSDMAgileProjectFramework>.

38. Diab, A.A.Z. Optimal shunt capacitors sittings and sizing in radial distribution systems using a novel hybrid optimization algorithm / A.A.Z. Diab, V.N. Tulsky, M.A. Tolba // 18th International Middle-East Power Systems Conference, MEPCON. – 2016. – No. 18. – P. 445–452.

39. Tolba, M.A. Comprehensive analysis of optimal allocation of capacitor banks in various distribution networks using different hybrid optimization algorithms / M.A. Tolba, V.N. Tulsky, A.S. Vanin, A.A.Z. Diab // Conference Proceedings – 17th IEEE International Conference on Environment and Electrical Engineering and 2017, 1st IEEE Industrial and Commercial Power Systems Europe, IEEEIC. – 2017. – No. 17. – P. 7977442.

40. Tolba, M.A. VLCI approach for optimal capacitors allocation in distribution networks based on hybrid PSOGSA optimization algorithm / M.A. Tolba, A.A.Z. Diab, V.N. Tulsky and A.Y. Abdelaziz // Electrical Engineering (Archiv für Elektrotechnik). – 2018. – Vol. 100. – No. 3. – P. 2059–2084.

41. Tolba, M.A. Integration of Renewable Distributed Generation in Distribution Networks Including a Practical Case Study Based on a Hybrid PSOGSA Optimization Algorithm / M.A. Tolba, A.S. Vanin, V.N. Tulsky, A.A.Zaki Diab, A.Y. Abdelaziz // Electric Power Components and Systems. – 2018. – Vol. 46. – No. 19–20. – P. 2103–2116.

42. Aljendy, R.I. Bio-inspired Optimization Techniques for Compensation Reactive Power and Voltage Profile Improvement in the Syrian Electrical Distribution Systems / R.I. Aljendy, R.R. Nasyrov, H.M. Sultan, A.A.Zaki Diab // Proceedings of the IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering, ElConRus, 2019. – P. 928–933.

43. GOST 28195-89. Assessment of software quality. General provisions [Electronic resource]. – Access mode : <https://docs.cntd.ru/document/1200009135>.

44. Ignatov, A.I. Selection of Geometric Parameters of an Reaction Wheel System Positioning when Controlling the Angular Motion of a Spacecraft / A.I. Ignatov // Journal of computer and systems

sciences international. – 2022. – Vol. 61. – No. 1. – P. 83–103.

45. Butenko, Yu.I. Prospective Scientific Research Trend Identification Methods (Based on the Analysis of Gas Fuel-Related Publications) / Yu.I. Butenko et. al. // Automatic documentation and mathematical linguistics. – 2022. – Vol. 56. – No. 1. – P. 11–25.

46. Wang, L.K. Imitation learning for coordinated human-robot collaboration based on hidden state-space models / L.K. Wang et. al. // Robotics and computer-integrated manufacturing. – 2022. – Vol. 76.

47. [Electronic resource]. – Access mode : <https://dmgur.ru/wp-content/uploads/2023/01/MethodologyComparisonEng.pdf>.

© Д.А. Гурьянов, Г.И. Афанасьев, А.Г. Афанасьев, 2024